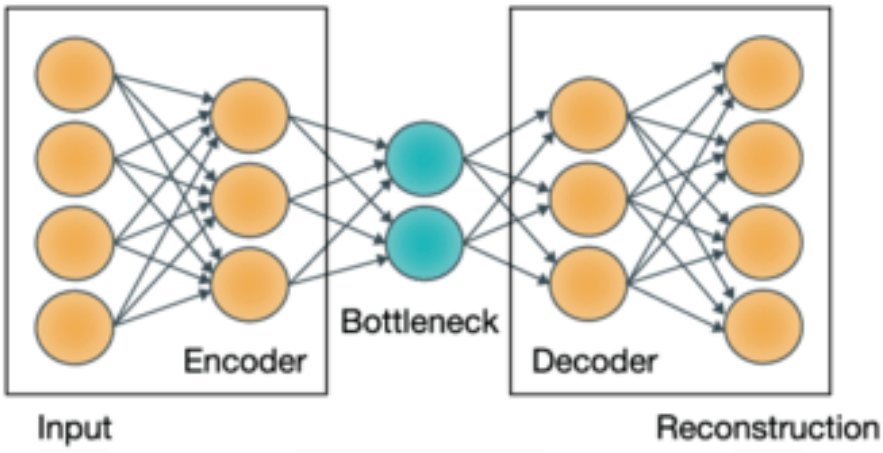
Lab 1 – MLP Autoencoder

Connor Johnston and Michael Greenspan

ELEC 475, Fall 2026

Part 2: In-Lab



Step 1 – Building a MLP Autoencoder

Move the Python files associated with the lab to your working project folder, alongside your prelab code files. The Python files are as follows:

- `model.py`
- `train.py`
- `test.py`

Train and test are already completed for you. The first stage of this lab will focus on building a 4-layer MLP autoencoder, which functions as follows: the encoder maps an unwrapped image down into a small bottleneck vector, and the decoder then maps that vector back to a full image.

Each fully connected (linear) layer has the following output shape:

1. `[1, N2]`
2. `[1, N_bottleneck]`
3. `[1, N2]`
4. `[1, N_output]`

Open `model.py`. The file includes 4 functions: `__init__`, `encode`, `decode`, `forward`.

`forward` is already completed for you. The other three methods must be completed.

`__init__`:
- Create 4 fully connected layers, for each output shape, using the provided parameters within the function.

`encode`:
- The first two fully connected layers will be in the encoder.
- After each layer, apply ReLU activation.

`decode`:
- The third and fourth fully connected layers will be in the decoder.
- After the third layer, apply ReLU activation.
- After the fourth layer, apply sigmoid activation, since pixel values fall between 0 and 1.

Before training, `train.py` prints a summary of your model. With a bottleneck size of 32, it should report 641,344 total parameters. If your number differs, correct your model before moving on.

Step 2 – Training

With the model completed, now it can be trained on the FashionMNIST dataset. The training process will take a few minutes to run on a CPU, and is a bit fast if using a GPU..

If using Colab, then you must change **DATA_DIR** in **train.py** to point to your Google Drive folder.

Run the following line in command prompt or terminal:

```
python train.py -z 32 -e 30 -b 256 -s MLP.32.pth -p loss.MLP.32.png
```

where the arguments refer to the following:

`-z` - Bottleneck size

`-e` - Number of epochs

`-b` - Batch size

`-s` - Save file (the trained weights are saved here after every epoch)

`-p` - Plot file. (loss curve is plotted here, review to confirm the model is training properly)

Step 3 – Testing

The model can now be tested on a separate set of data that it was not trained on.

If using Colab, then you must change **DATA_DIR** in **test.py** to point to your Google Drive folder.

Run the following line in command prompt or terminal:

```
python test.py -z 32 -s MLP.32.pth
```

where the **`-z`** hyperparameter must match the value used during training.

The test code will ask for an index. Enter indices within range to visualize the output. View several reconstructions across different classes. Notice that the overall shape survives, but with a bottleneck of just 32, fine details do not.

Step 4 – Image Denoising

Autoencoders can remove noise from images without being explicitly trained to do so.

- Begin by copying your **`test.py`** file and call it **`denoise.py`**.
- Modify this file to add random noise to each image, before passing it to the model.
- To generate random noise, use the **`torch.rand()`** method, with the noise scaled by about 0.3. Add the noise after the image has been normalized and clamp the result back to [0, 1] with **`torch.clamp()`**.
- Edit the matplotlib plot to display three images side by side: the original, the noisy input, and the reconstructed image.
- Run the file to view the output for various image classes.

Step 5 - Completion

To have your in-lab work marked as complete, you must show a TA the following:

- **`model.py`**
- **`loss.MLP.32.png`**
- **`MLP.32.pth`**
- Output from **`test.py`**
- Output from **`denoise.py`**

The marking rubric is as follows:

Item	mark
Correct **`model.py`**	0.5
Correct **`loss.MLP.32.png`**	0.5
`test.py` Output	1
`denoise.py` Output	1
Total:	**3**

The remaining lab work (and 4 marks) will be part of the post-lab, which is due after this lab session.